

Lab 3: Architectural Pattern Selection - Justification

Second-Hand Product Escrow Marketplace | Software Engineering Lab

Architecture Selection

We chose Microservices Architecture for the Second-Hand Product Escrow Marketplace System. The system is split into independently deployable services (Order Manager, Payment, Escrow, Inspection, Product Catalog, User & Auth, Notification) behind an API Gateway, each exposing a provided interface and consuming others through required interfaces, as shown in the component diagram.

Comparison of Considered Styles

Style	Fit for this marketplace
Layered	Simple, but payment, escrow and inspection logic would sit in one deployable business layer; one change redeploys everything and sensitive code cannot be isolated.
Client-Server	Central control and easy consistency, but a single server is a single point of failure and a bottleneck, which is risky for a service that holds user money.
Microservices	Best fit: independent workflows (order, payment, escrow, inspection) with different security and load profiles.

Two Reasons for the Choice

1. Escrow, payment and inspection are separate business capabilities with their own lifecycle. Per FR-002 to FR-005, funds are held in escrow while the order is 'Under Inspection', released on Buyer approval, and frozen on dispute. Keeping Escrow Service, Payment Service and Inspection Service as separate components means the escrow rules can change (for example, new dispute handling) without redeploying the Order or Catalog services, and the External Payment Gateway can be replaced by any provider offering the same *IPaymentProcessor* interface.

2. The three user roles create uneven, independent workloads. Buyers browse the catalogue constantly (FR-001), while Inspection Agents submit reports far less often (FR-003). Separate services let the read-heavy Product Catalog scale independently of the Escrow ledger, and a failure in the Notification Service does not stop a Buyer from paying or an Agent from submitting a report, because notifications are sent asynchronously.

Security Advantage (NFR-001)

All external traffic enters through a single API Gateway that terminates TLS and validates the JWT/OAuth2 token issued by the User & Auth Service, so no internal service is directly exposed. Card data is handled only by the Payment Service, which talks to the external processor over HTTPS using tokenised data, and the Escrow ledger is a separate service with its own datastore. A compromise of a low-risk service such as Product Catalog or Notification therefore does not give direct access to payment or escrow data (fault and breach isolation).

Performance Benefit (NFR-002)

NFR-002 requires 95% of requests to be answered within 3 seconds. With microservices, only the busy services (mainly Product Catalog and Order Manager) need to be scaled out or cached under peak load, rather than the whole application. Slow, non-critical work (notifications) is moved to asynchronous messaging so it never blocks the Buyer's or Inspection Agent's request, keeping response times within the target.

Trade-off Acknowledged

Microservices add operational complexity, network latency between services and data-consistency challenges. We accept these because the escrow flow needs strong isolation, and we limit the risk by using a gateway, a clear interface per service and a database per service.